



CE 206: Engineering Computation Sessional

MATLAB Fundamentals

Department of Civil Engineering

Course Outline

Introduction to hi-level computational programming tools

- MATLAB, Mathematica etc.

Application to numerical analysis

- Basic matrix computations
- Solving system of linear equations
- Solving non-linear equations
- Solving differential Equations
- Interpolation and curve-fitting
- Numerical differentiation
- Numerical integration

Application to engineering problems

- Solving mechanics problems
- Numerical solution of equation of motion etc.

Course Layout

Class Assessment

- Approximately one per week
- Solving a problem in a given amount of time
- Print out the codes and figures and keep it in file

Assignment/Homework/File

Mid-term Quiz

- Solving problems in a given amount of time

Project

- Group submission

Final Quiz

References and Resources

Requirements

- Basic familiarity with programming (e.g. CE204)
- Knowledge on linear algebra, differential equations and numerical methods (CE 205)

The MATLAB help file documentation

- Provided with the MATLAB software
- www.mathworks.com

Materials to be provided in the class

You will also need:

- Your CE 205 Course references/Lecture slides

Introduction to MATLAB

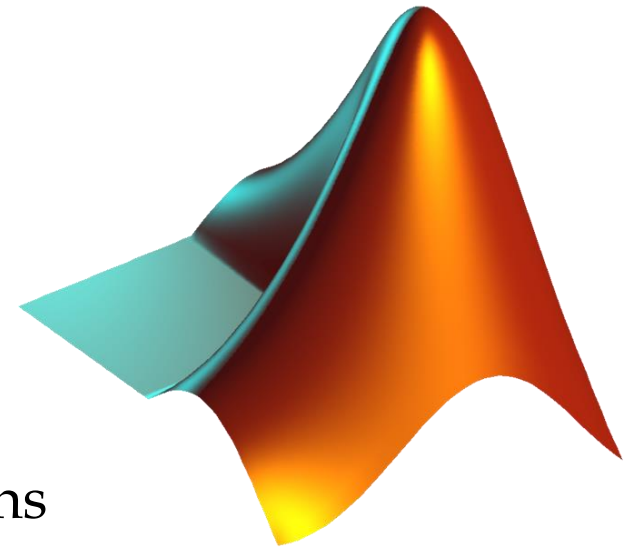
MATLAB (MATrix LABoratory) is a fully-functional programming language

Original code written by Cleve Moler of UNM in the 1970s, later released as a commercial package by Mathworks, Inc.

Originally intended for interactive numerical computations

Interfaces with programs written in other languages (C++, Fortran)

Widely used in academia, research institutions and industry worldwide



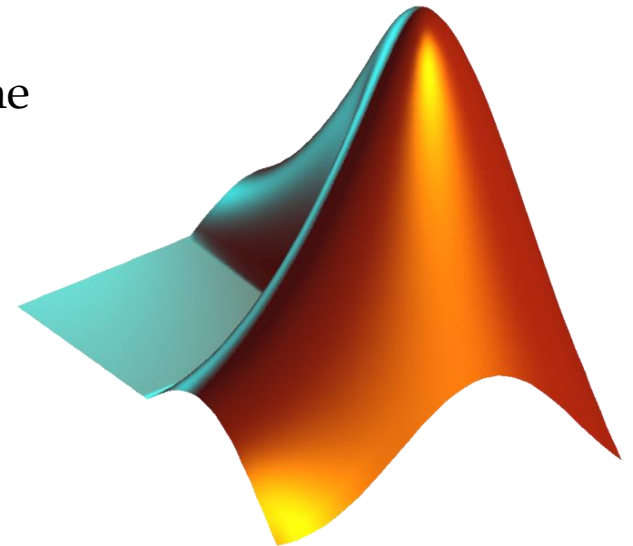
MATLAB basics

MATLAB can be thought of as a super-powerful graphing calculator with many more buttons (i.e. built-in functions)

You can build up your own set of functions suited for a particular operation

It is an interpreted programming language
-commands are executed line by line

It is capable of graphically representing computational results simultaneously.
- Not possible in C++, Fortran



MATLAB Installation

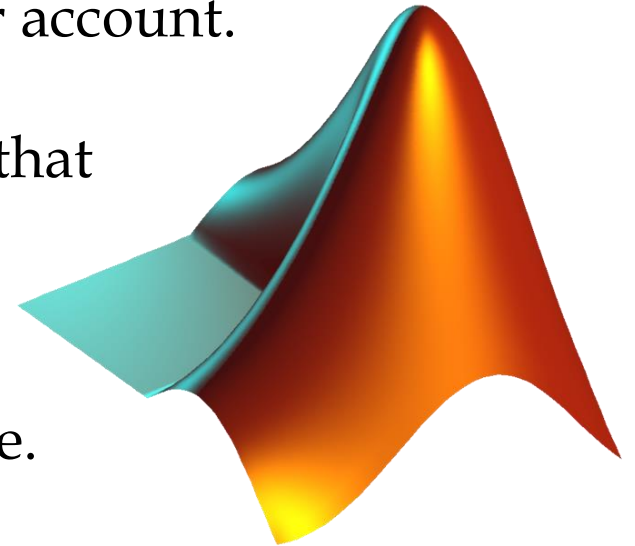
Licensed version of MATLAB is accessible with buet.ac.bd email id.

Create an account in <https://www.mathworks.com/> with your “*@ce.buet.ac.bd” email id.

License will be automatically added to your account.

Log in into your mathworks account using that same email id from inside your installed MATLAB software.

Update the license obtained with this license.



MATLAB as a calculator

Basic arithmetic operators (+, -, *, /) used in conjunction with brackets ()

<pre>>> -5 / (4.8+5.32)^2</pre>	→	$-\frac{5}{(4.8+5.32)^2}$
<pre>ans = -0.0488</pre>		
<pre>>> (3+4i) * (3-4i)</pre>	→	$(3+4i)(3-4i)$
<pre>ans = 25</pre>		
<pre>>> cos(pi/2)</pre>	→	$\cos(\pi/2)$
<pre>ans = 6.1230e-017</pre>		
<pre>>> exp(acos(0.3))</pre>	→	$e^{\cos^{-1}(0.3)}$
<pre>ans = 3.5470</pre>		

Built-in functions

<code>sin()</code>	<code>asin()</code>
<code>cos()</code>	<code>acos()</code>
<code>tan()</code>	<code>atan()</code>
<code>log()</code>	<code>log10()</code>
<code>exp()</code>	<code>sqrt()</code>
<code>abs()</code>	<code>round()</code>

Numbers and Formats

Type	Examples
Integer	1362, -217897
Real	1.234, -10.76
Complex	$3.21 - 4.3i$ ($i = \sqrt{-1}$)
Inf	Infinity (result of dividing by 0)
NaN	Not a Number, 0/0

Command	Example of Output
<code>>>format short</code>	31.4162(4-decimal places)
<code>>>format short e</code>	3.1416e+01
<code>>>format long e</code>	3.141592653589793e+01
<code>>>format short</code>	31.4162(4-decimal places)
<code>>>format bank</code>	31.42(2-decimal places)

All computations in MATLAB are done in double precision (15 significant figures)

Variables

- Variable names and their types do not have to be declared in MATLAB.
- If a statement is terminated with a semicolon (;), the results are suppressed
- Variable names must start with a letter followed by letters, digits, and underscores.
- The name of variable is not accepted if it is reserved word.

Example

```
>> x=-13; y = 5*x, z = x^2+y  
y =  
-65  
z =  
104
```

Variables

Commands involving variables

- **who**: lists the names of the defined variables
- **whos**: lists the names and sizes of defined variables
- **clear**: clears all variables
- **clear** *name*: clears the variable *name*
- **clc**: clears the command window

Avoid using

- **ans**: default variable name for the result.
- **pi**: $\pi = 3.1415926 \dots$
- **eps**: $\varepsilon = 2.2204e-016$, smallest value by which two numbers can differ
- **inf**: ∞ , infinity
- **NAN** or **nan**: not-a-number

Vectors

- Entries within a row are separated by spaces or commas.
- Rows are separated by semicolons.
- Vector properties using **size()** and **length()**

```
>> a = [1 2 3 4 5] → col=5  
a =  
      1      2      3      4      5  
>> b = [2;4;6;8;10] → row=5  
b =  
      2  
      4  
      6  
      8  
     10  
      col=1
```

```
>> size(a)  
ans =  
      1      5  
  
>> length(a)  
ans =  
      5
```

Other methods of vector creation

The colon operator: generally **a : b : c** produces a vector of entries starting with the value **a**, incrementing by the value **b** until it gets to **c**

```
>> 3:7
ans =
     3     4     5     6     7

>> 0.32:0.1:0.6
ans =
0.3200 0.4200 0.5200

>> -1.4:-0.3:-2
ans =
-1.4000 -1.7000 -2.0000
```

linspace (a,b,n)

generates n equispaced points between a and b, inclusive.

Matrices

A 2-D array, or matrix, of data is entered row by row, with spaces (or commas) separating entries within the row and semicolons separating the rows:

```
>> A = [1 2 3; 4 5 6; 7 8 9]
```

1st row *2nd row* *3rd row*

A =

1	2	3
4	5	6
7	8	9

Extracting bits of matrices:

A(j,k) gives j'th row, k'th column

A(2:3,1:2) rows 2 through 3 and columns 1 through 2

A([1,3], :) all of rows 1 and 3

A(:, 1) first column

Numerical Array Concatenation

```
>> a=[1 2;3 4]
a =
     1     2
     3     4
>> cat_a=[a 2*a; 3*a 4*a; 5*a 6*a]
cat_a =
     1     2     2     4
     3     4     6     8
     3     6     4     8
     9    12    12    16
     5    10     6    12
    15    20    18    24
```

Use square brackets []

4*a

Use [] to combine existing arrays as matrix “elements”

More on Matrices

zeros (n)	Returns a $n \times n$ matrix of zeros
zeros (m, n)	Returns a $m \times n$ matrix of zeros
rand (m, n)	Returns a $m \times n$ matrix of random numbers
eye (m, n)	Returns a $m \times n$ Identity matrix
ones (n)	Returns a $n \times n$ matrix of ones
ones (m, n)	Returns a $m \times n$ matrix of ones
size (A)	For a $m \times n$ matrix A, returns the row vector [m,n] containing the number of rows and columns in matrix
length (A)	Returns the larger of the number of rows or columns in A

Diagonal Matrix

First define a vector containing the values of the diagonal entries (in order) then use the **diag** function to form the matrix

```
>> d = [-3 4 2], D = diag(d)
```

```
d =
```

```
-3 4 2
```

```
D =
```

```
    -3  0  0
```

```
     0  4  0
```

```
     0  0  2
```

This command is useful when we need to construct very large matrices.

If A is a matrix, $\text{diag}(A)$ extracts the diagonal elements of the matrix.

Sparse Matrix

These are generally large matrices that have a very small proportion of non-zero entries

```
>> i = [1, 3, 5]; j = [2,3,4];  
>> v = [10 11 12];  
>> S = sparse (i,j,v)
```

```
S =
```

```
(1,2) 10
```

```
(3,3) 11
```

```
(5,4) 12
```

```
>> T = full(S)
```

```
T =
```

```
0 10 0 0
```

```
0 0 0 0
```

```
0 0 11 0
```

```
0 0 0 0
```

```
0 0 0 12
```

Creating a 5-by-4 sparse matrix S having only 3 non-zero values:

$S(1,2) = 10,$

$S(3,3) = 11$ and

$S(5,4) = 12$

The “full” command displays the sparse matrix

Matrix operations

Transpose	$B = A'$
Addition and Subtraction	$C = A+B \quad C = A-B$
Scalar Multiplication	$B = \alpha A$, where α is a scalar
Matrix Multiplication	$C = A * B$
Matrix Inverse	$B = \text{inv}(A)$, A must be a square matrix
Matrix powers	$B = A * A$, A must be a square matrix
Determinant	$\det(A)$, A must be a square matrix

Operators $*$ $^{\wedge}$ and $/$ have two modes of operation

- standard
- element-wise

Standard matrix product operation

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} * \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} = 11$$

$$1 \times 3 * 3 \times 1 = 1 \times 1$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 9 \\ 6 & 12 & 18 \\ 9 & 18 & 27 \end{bmatrix}$$

$$3 \times 3 * 3 \times 3 = 3 \times 3$$

$$A = m \text{ rows} \left\{ \begin{bmatrix} \text{---} \\ \text{---} \\ \vdots \\ \text{---} \end{bmatrix} \right\}, \quad B = \underbrace{\begin{bmatrix} | & | & \dots & | \end{bmatrix}}_{p \text{ columns}}$$

$$(m \times \boxed{n}) \text{ times } (\boxed{n} \times p) \Rightarrow (m \times p).$$

Elementwise matrix operations

To do element-wise operations, use the dot $.(*, ./, .^)$. **BOTH dimensions must match (unless one is scalar)**

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} .* \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} = \text{ERROR}$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} .* \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 3 \end{bmatrix}$$
$$3 \times 1 .* 3 \times 1 = 3 \times 1$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix} .* \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

$$3 \times 3 .* 3 \times 3 = 3 \times 3$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} .^2 = \begin{bmatrix} 1^2 & 2^2 \\ 3^2 & 4^2 \end{bmatrix}$$

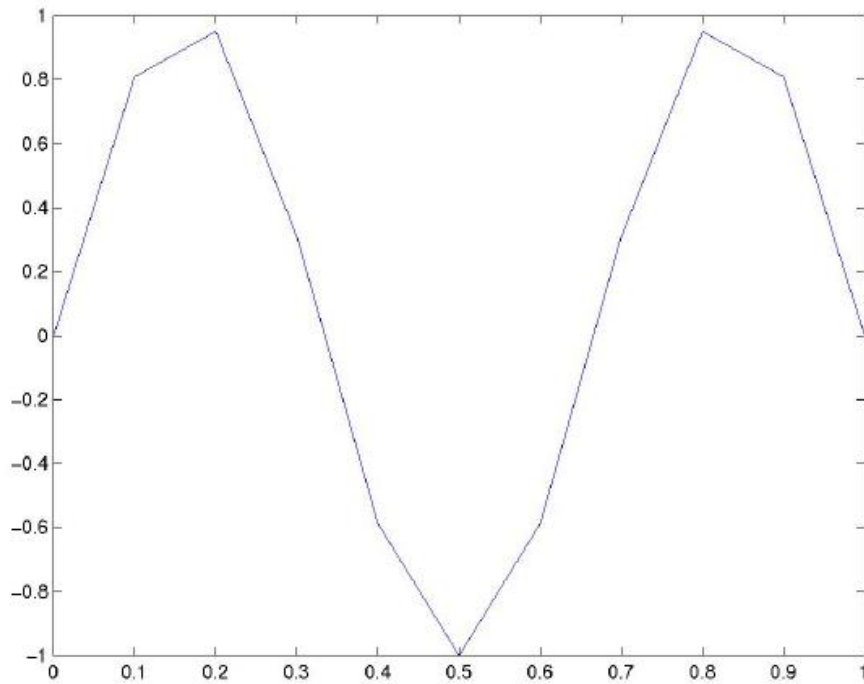
Can be any dimension

Graphics: plotting functions

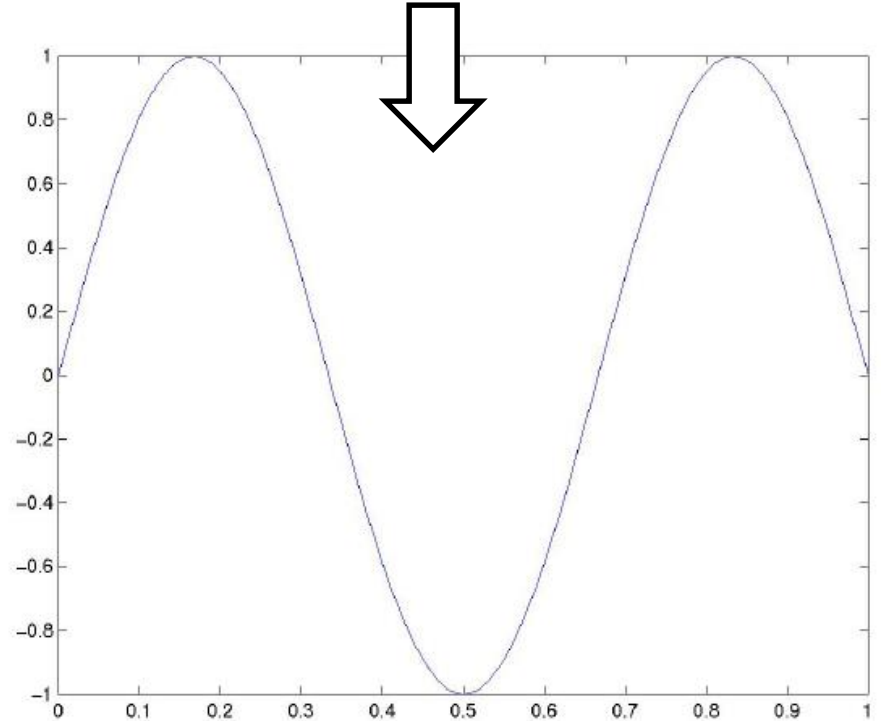
$$y = \sin 3\pi x$$

$$\text{for } 0 \leq x \leq 1$$

```
>> x = linspace (0,1,11);  
>> y = sin(3*pi*x);  
>> plot(x,y)
```

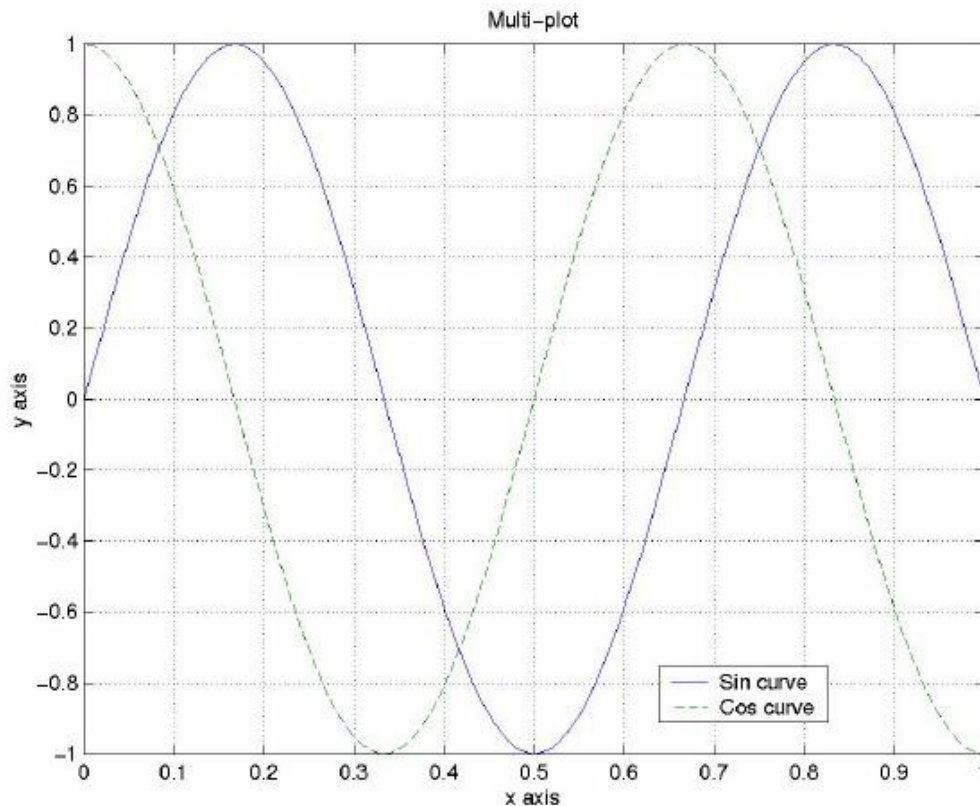


Increasing the number of elements in x



Multiplots, titles, labels, linestyle and colors

```
>> plot(x,y,'b-',x,cos(3*pi*x),'g--')
>> legend('Sin curve','Cos curve')
>> title('Multi-plot ')
>> xlabel('x axis'), ylabel('y axis')
>> grid
```



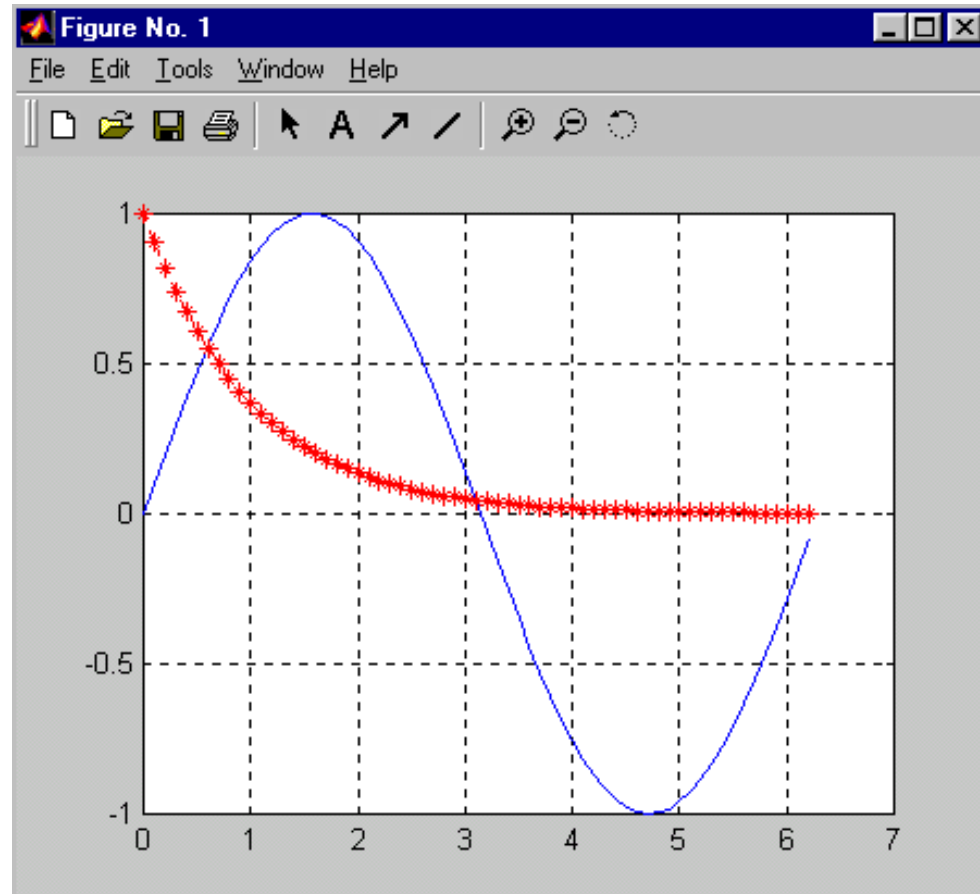
Colours		Line Styles	
y	yellow	.	point
m	magenta	o	circle
c	cyan	x	x-mark
r	red	+	plus
g	green	-	solid
b	blue	*	star
w	white	:	dotted
k	black	-.	dashdot
		--	dashed

Adding additional plots

■ hold on and hold off

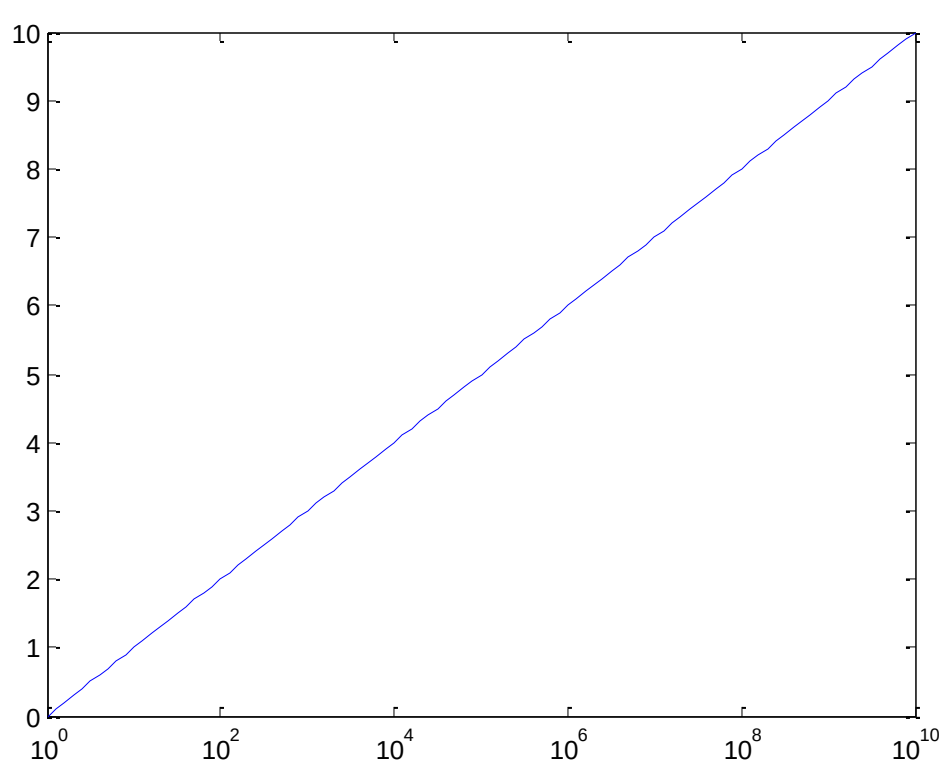
hold on tells MATLAB to keep the current data plotted and add the results of any further plot commands to the graph. Hold off releases the hold on the figure

```
» x = 0:.1:2*pi;  
» y = sin(x);  
» plot(x,y,'b')  
» grid on;  
» hold on;  
» plot(x,exp(-x), 'r*');
```

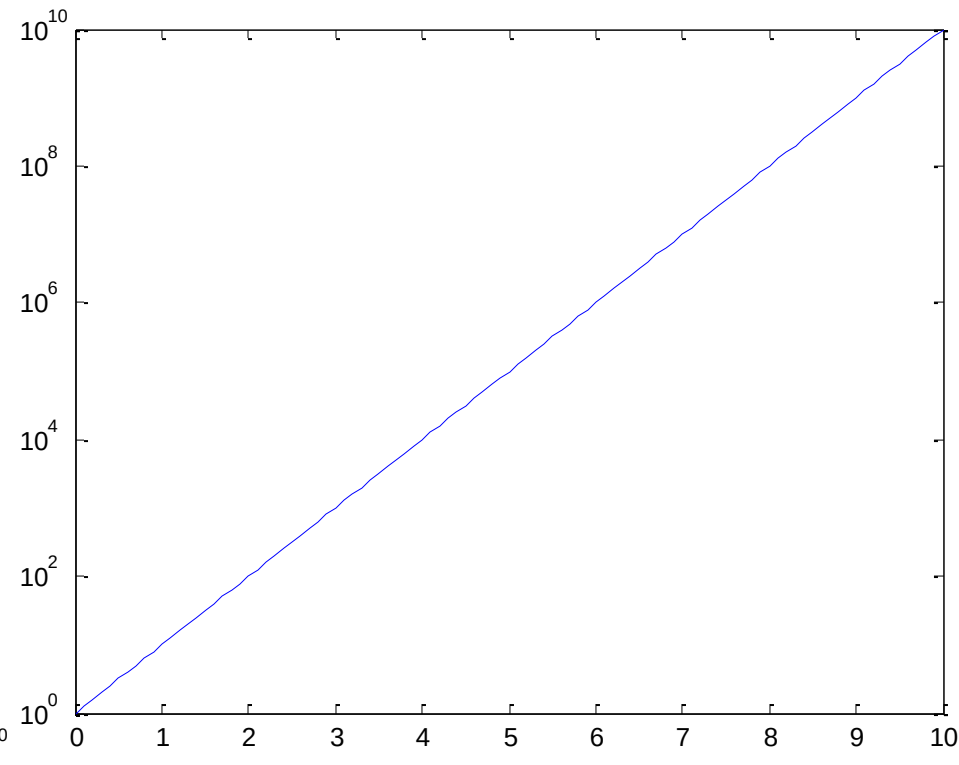


Other plotting commands: **semilogx**, **semilogy**

```
x = 0:0.1:10;  
semilogx(10.^x,x)
```

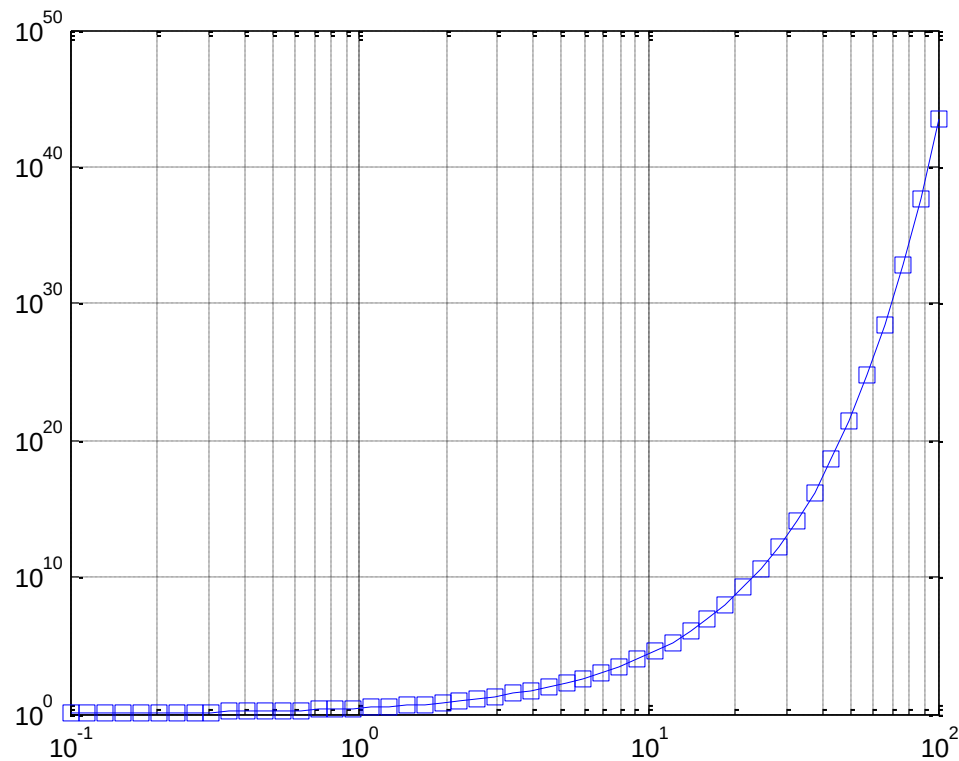


```
x = 0:0.1:10;  
semilogy(x, 10.^x)
```



Other plotting commands: **loglog**

```
x = logspace(-1,2);  
loglog(x,exp(x),'s-')  
grid on
```



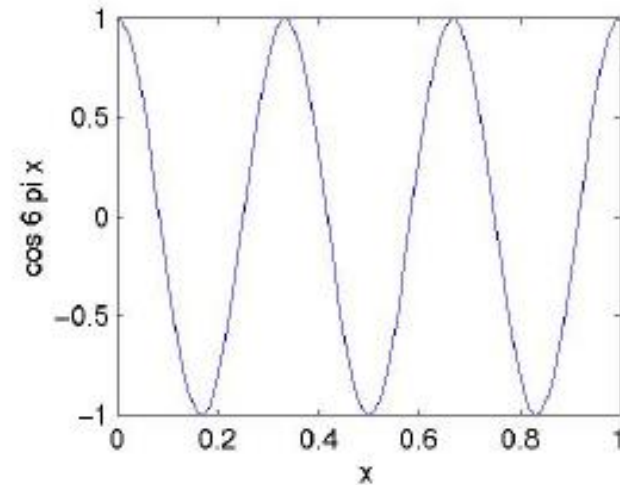
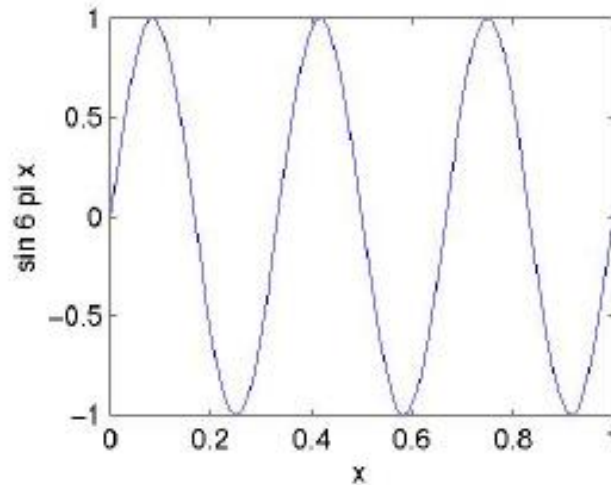
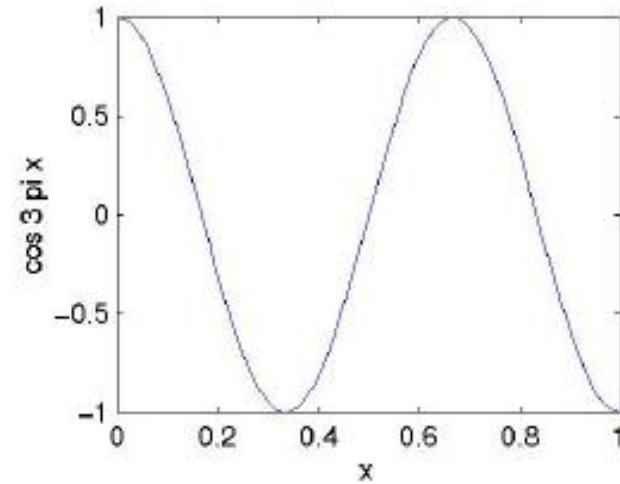
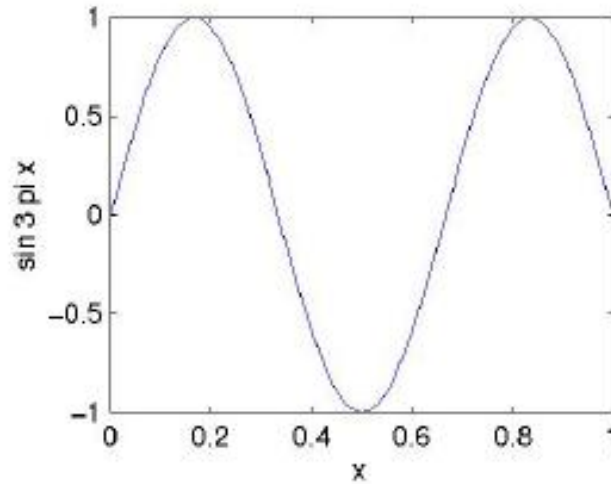
Subplot example

```
>> subplot(221), plot(x,y)
>> xlabel('x'),ylabel('sin 3 pi x')
>> subplot(222), plot(x,cos(3*pi*x))
>> xlabel('x'),ylabel('cos 3 pi x')
>> subplot(223), plot(x,sin(6*pi*x))
>> xlabel('x'),ylabel('sin 6 pi x')
>> subplot(224), plot(x,cos(6*pi*x))
>> xlabel('x'),ylabel('cos 6 pi x')
```

■ `subplot(m, n, p)`

subplot splits the figure window into an $m \times n$ array of small axes and makes the p^{th} one active. Note - the first subplot is at the top left, then the numbering continues across the row

Subplot example



Formatted text on plots

plot the first 100 terms in the sequence $\{x_n\}$ given by $x_n = [1 + 1/n]^n$ and then graph the function $\Phi(x) = x^3 \sin^2(3\pi x)$ on the interval $-1 \leq x \leq 1$

```
>> set(0,'Defaultaxesfontsize',16);  
>> n = 1:100; x = (1+1./n).^n;  
>> subplot (211)  
>> plot(n,x,'.',[0 max(n)],exp(1)*[1 1],...  
    '--','markersize',8)  
>> title('x_n = (1+1/n)^n','fontsize',12)  
>> xlabel('n'), ylabel('x_n')  
>> legend('x_n','y = e^1 = 2.71828...')  
>> %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
>> subplot (212)  
>> x = -2:.02:2; y = x.^3.*sin(3*pi*x).^2;  
>> plot(x,y,'linewidth',2)  
>> legend('\phi(x) = x^3 sin^2 3\pi x')  
>> xlabel('x')
```

Default font
changed

Markersize
changed

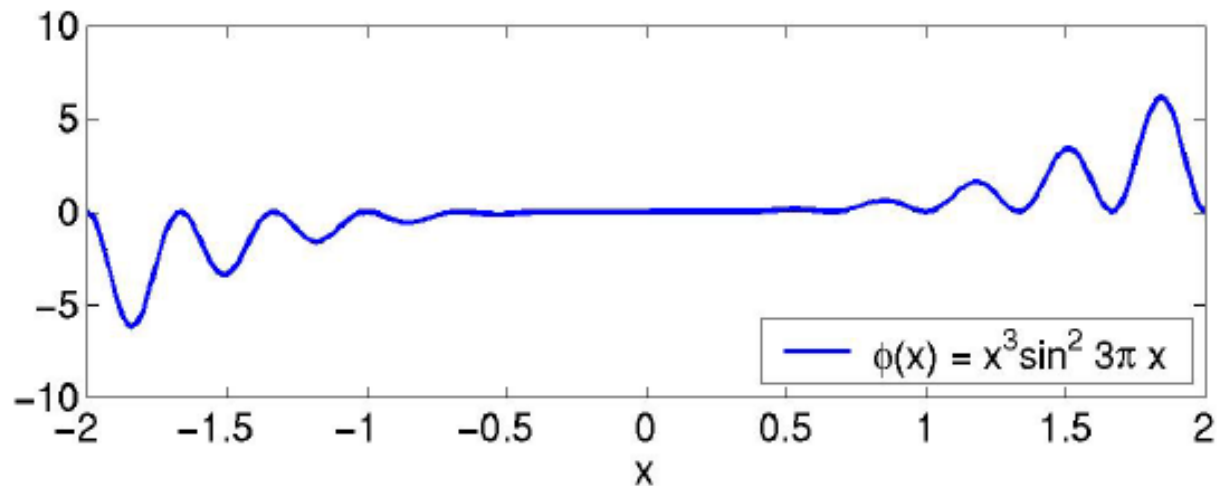
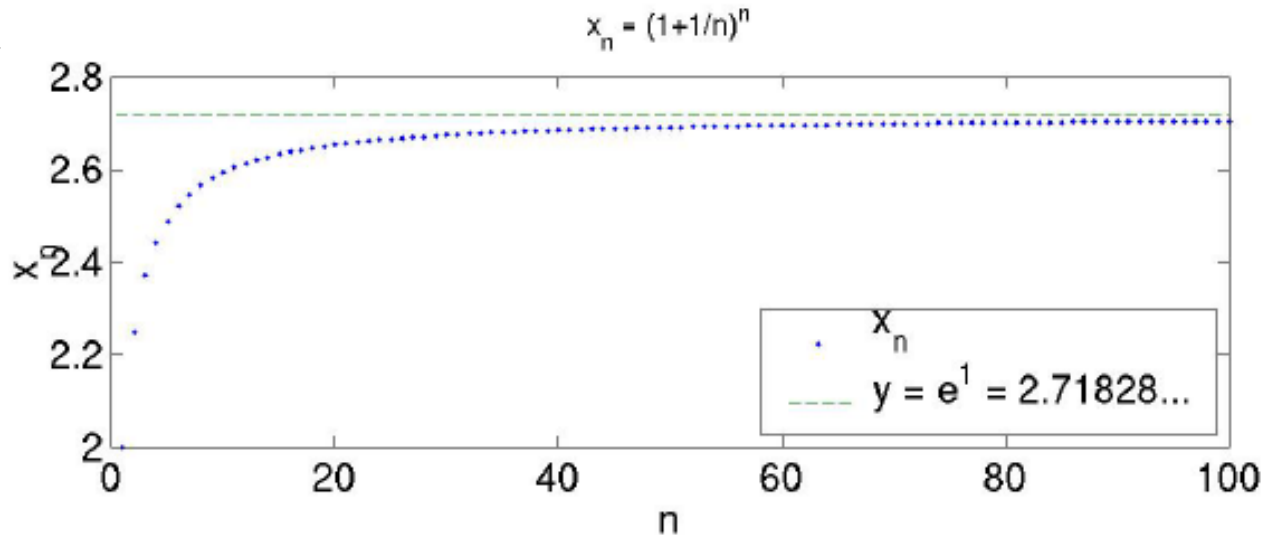
Subscript/
superscript

LineWidth
changed

Latin
characters

Formatted text on plots

plot the first 100 terms in the sequence $\{x_n\}$ given by $x_n = [1 + 1/n]^n$ and then graph the function $\Phi(x) = x^3 \sin^2(3\pi x)$ on the interval $-1 \leq x \leq 1$

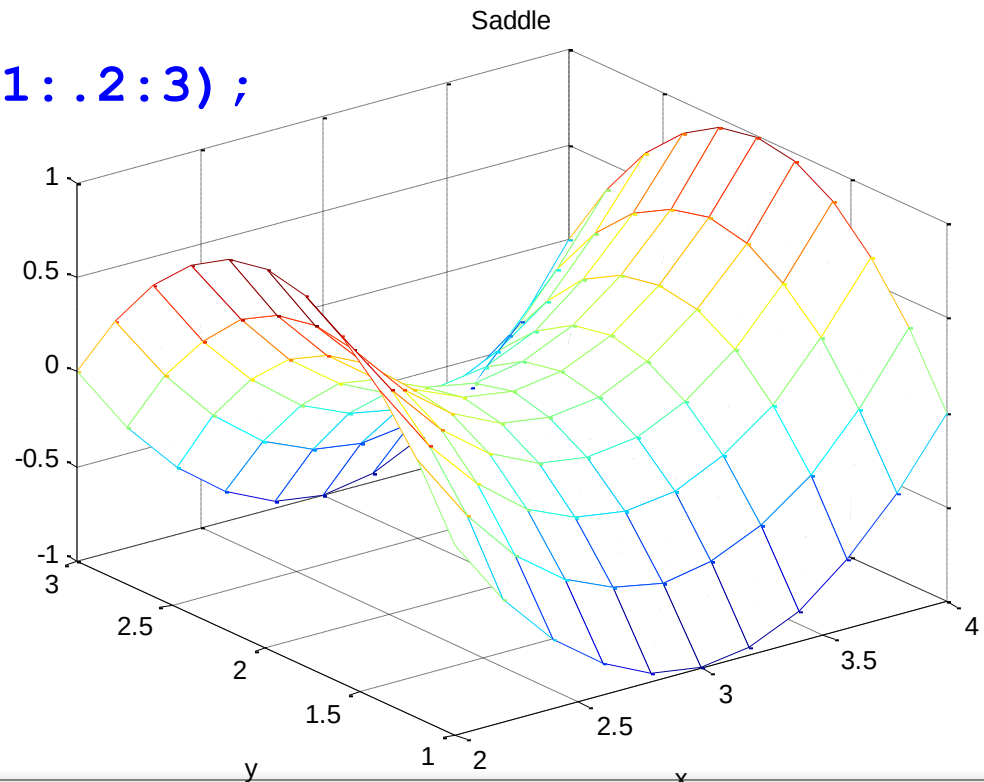


Advanced graphics: plotting surfaces

- The command **meshgrid** is used to construct the (x, y) gridpoints at certain intervals
- Evaluate the function $z = f(x,y)$ at all the gridpoints
- Use a surface plot feature (e.g. **mesh**, **surf**) to plot the surface

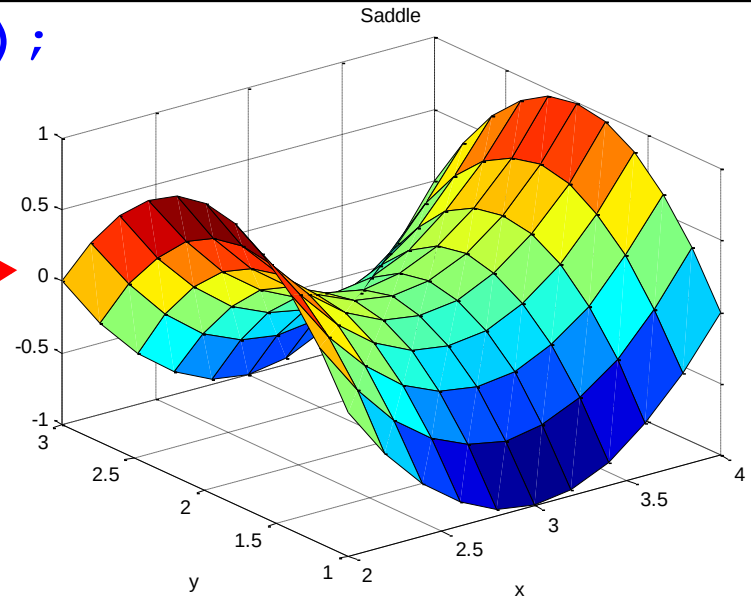
Example: Plot the surface defined by $z = (x - 3)^2 + (y - 2)^2$ for $2 \leq x \leq 4$ and $1 \leq y \leq 3$

```
[X,Y] = meshgrid(2:.2:4, 1:.2:3);  
Z = (X-3).^2 - (Y-2).^2;  
mesh(X,Y,Z)  
title('Saddle'),  
xlabel('x'),ylabel('y')
```



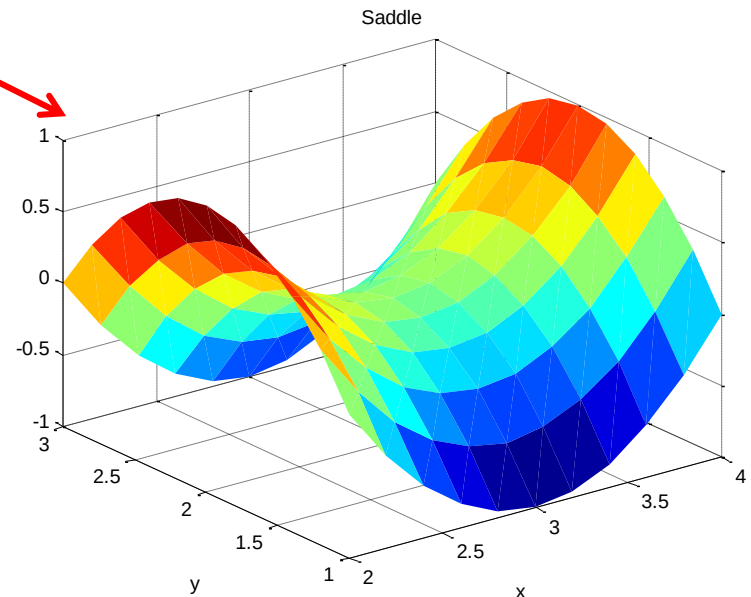
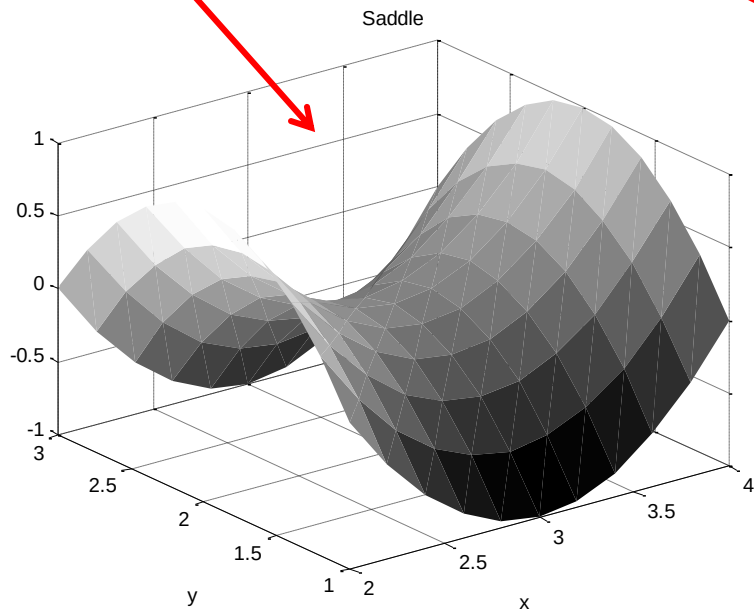
Advanced graphics: **surf** and **shading**

```
[X,Y] = meshgrid(2:.2:4, 1:.2:3);  
Z = (X-3).^2 - (Y-2).^2;  
surf(X,Y,Z)  
title('Saddle')  
shading faceted
```



shading flat

colormap(gray)

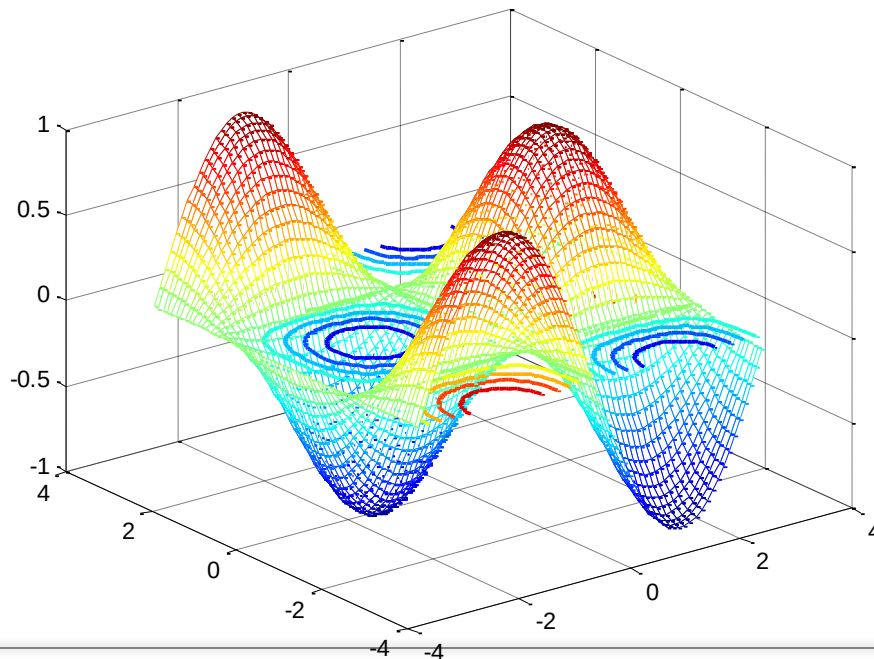
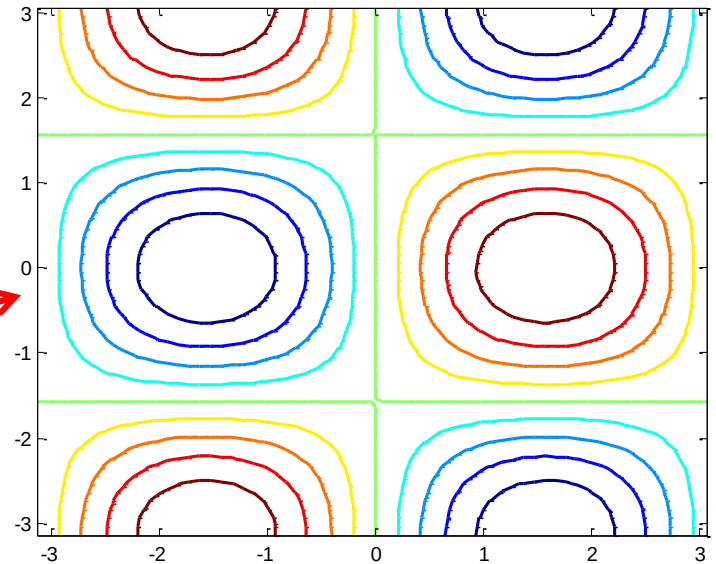


Advanced graphics: **contour** plot

Takes the same arguments as **surf** or **mesh**

Example: Plot the contour of the surface defined by $z = (\sin x)(\cos y)$ for $-\pi \leq x \leq \pi$ and $-\pi \leq y \leq \pi$

```
x=-pi:0.1:pi;  
y=-pi:0.1:pi;  
[X,Y]=meshgrid(x,y);  
Z =sin(X) .*cos (Y);  
contour (X,Y,Z, 'LineWidth',2)
```



```
mesh(X,Y,Z); hold on;  
contour(X,Y,Z)
```

Specialized plotting functions

polar-to make polar plots

```
polar(0:0.01:2*pi,cos((0:0.01:2*pi)*2))
```

bar-to make bar graphs

```
bar(1:10,rand(1,10));
```

quiver-to add velocity vectors to a plot

```
[X,Y]=meshgrid(1:10,1:10);  
quiver(X,Y,rand(10),rand(10));
```

stairs-plot piecewise constant functions

```
stairs(1:10,rand(1,10));
```

fill-draws and fills a polygon with specified vertices

```
fill([0 1 0.5],[0 0 1],'r');
```

for loop

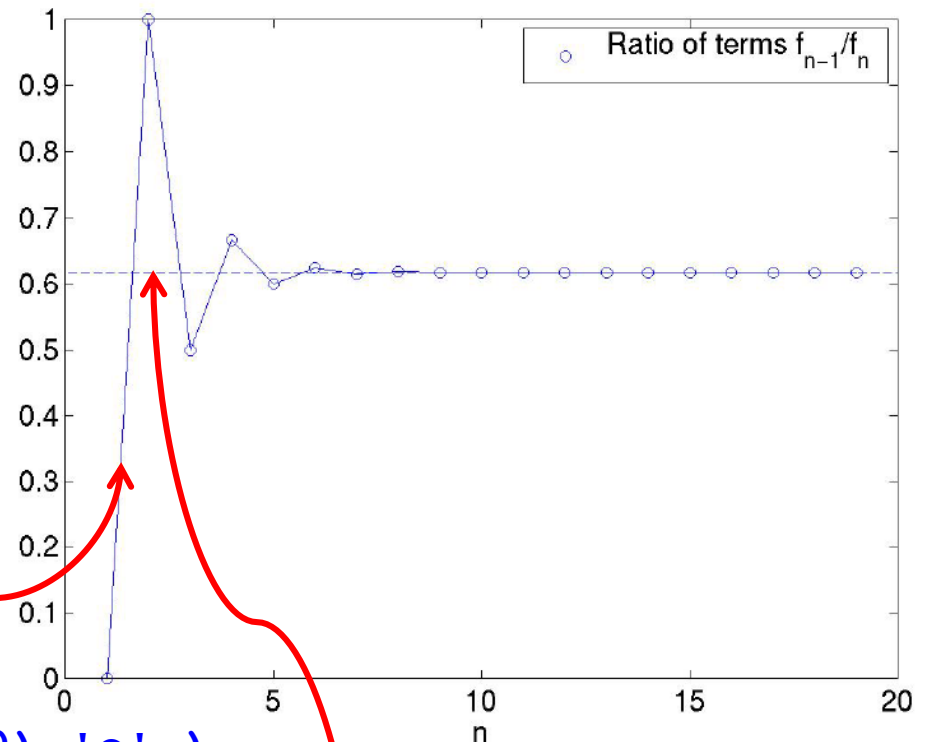
Used when we want to repeat a segment of the code several times

Example: Test the assertion that the ratio of the two successive values in the Fibonacci series approach the golden ratio of $(\sqrt{5}-1)/2$

. i.e. $f_n/f_{n-1} = (\sqrt{5}-1)/2$ where $n = 3, 4, 5$

.....

```
>> F(1) = 0; F(2) = 1;  
>> for i = 3:20  
F(i) = F(i-1) + F(i-2);  
end  
>> plot(1:19, F(1:19) ./ F(2:20), 'o' )  
>> hold on, xlabel('n')  
>> plot(1:19, F(1:19) ./ F(2:20), '-' )  
>> legend('Ratio of terms f_{n-1}/f_n')  
>> plot([0 20], (sqrt(5)-1)/2*[1,1], '--')
```



while loop

when we want to repeat a segment of the code several times until some logical condition is satisfied.

The while loop is used when we do not know for certain how many iterations may be needed

Example: What is the greatest value of n that can be used in the sum

$$1^2 + 2^2 + \cdots + n^2$$

and get a value less than 100?

```
>> S = 1; n = 1;  
>> while S+ (n+1)^2 < 100  
n = n+1; S = S + n^2;  
end  
>> [n, S]  
ans =  
6 91
```

Logical and relational operators

<code>==</code>	Equal
<code>~=</code>	Not equal
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater or equal
<code><=</code>	Less or equal
<code>& &&</code>	And
<code> </code>	Or

```
x =  
-2.0000  3.1416  5.0000  
-5.0000 -3.0000 -1.0000  
>> x > 3 & x < 4  
ans =  
0 1 0  
0 0 0  
>> x > 3 | x == -3  
ans =  
0 1 1  
0 1 0
```

Matlab represents true and false by means of the integers ¹0 and ⁰1

Flow control using `if/else/elseif`

IF

```
if cond
    commands
end
```

Conditional statement:
evaluates to true or false

ELSE

```
if cond
    commands1
else
    commands2
end
```

ELSEIF

```
if cond1
    commands1
elseif cond2
    commands2
else
    commands3
end
```

```
count=0;
for n=1:length(x)
    if x(n)>0
        count=count+1;
    end
end
```

Example: Given
 $x = \sin(\text{linspace}(0, 10\pi, 100))$, how many
of the entries are positive?

The **find** command for vectors

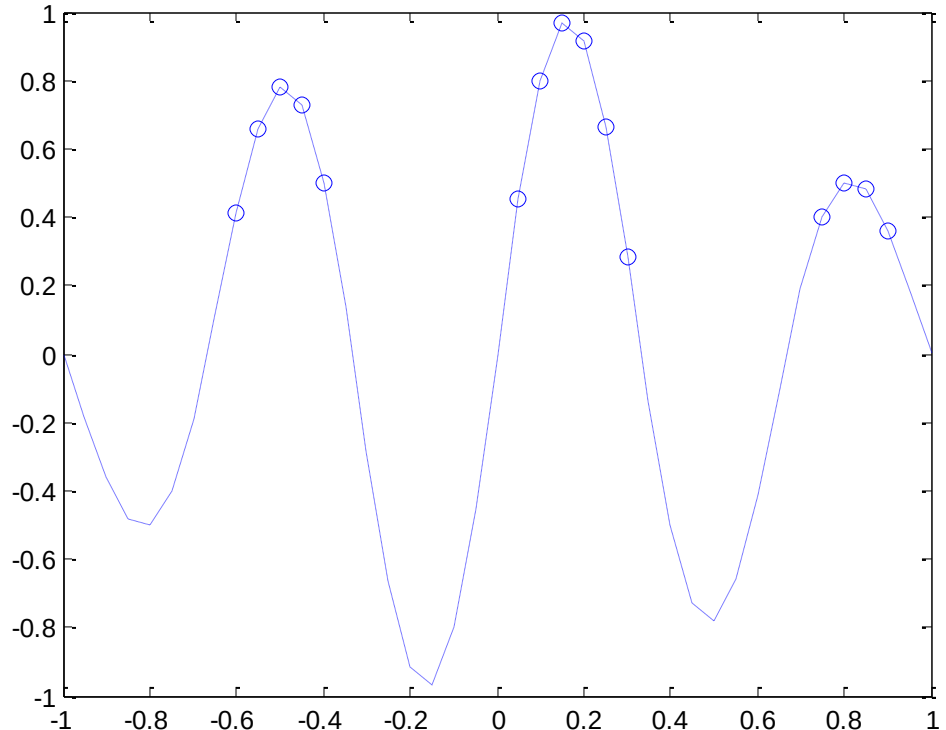
returns a list of the positions (indices) of the elements of a vector satisfying a given condition.

Example: Produce a plot for $y = e^{-x^2} \sin(3\pi x)$ and mark all the points that have a value of y greater than 0.2

```
>> x = -1:.05:1;  
>> y = sin(3*pi*x) .* exp(-x.^2) ;  
>> k = find(y > 0.2)
```

```
k =  
Columns 1 through 12  
9 10 11 12 13 22 23  
24 25 26 27 36  
Columns 13 through 15  
37 38 39
```

```
>> plot(x,y,':'); hold on;  
>> plot(x(k),y(k),'o')
```

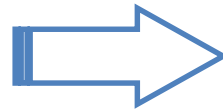


Avoiding loops: efficient coding

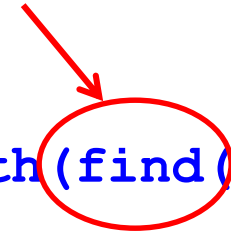
Example 1: Given $x = \sin(\text{linspace}(0, 10\pi, 100))$, how many of the entries are positive?

```
count=0;  
for n=1:length(x)  
    if x(n)>0  
        count=count+1;  
    end  
end
```

Using the “find” command

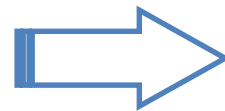


```
Count=length(find(x>0));
```



Example 2 find the sum: $1^2 + 2^2 + 3^2 + \dots + 100^2$

```
sum_sq=0;  
for n=1:100  
    sum_sq = sum_sq + n^2;  
end
```



```
Sum_sq=sum((1:100).^2)
```

Vectorization



Avoiding loops: efficient coding

Built-in functions (e.g. find, sum) will make it faster to write and execute

```
count=0;  
for n=1:length(x)  
    if x(n)>0  
        count=count+1;  
    end  
end
```

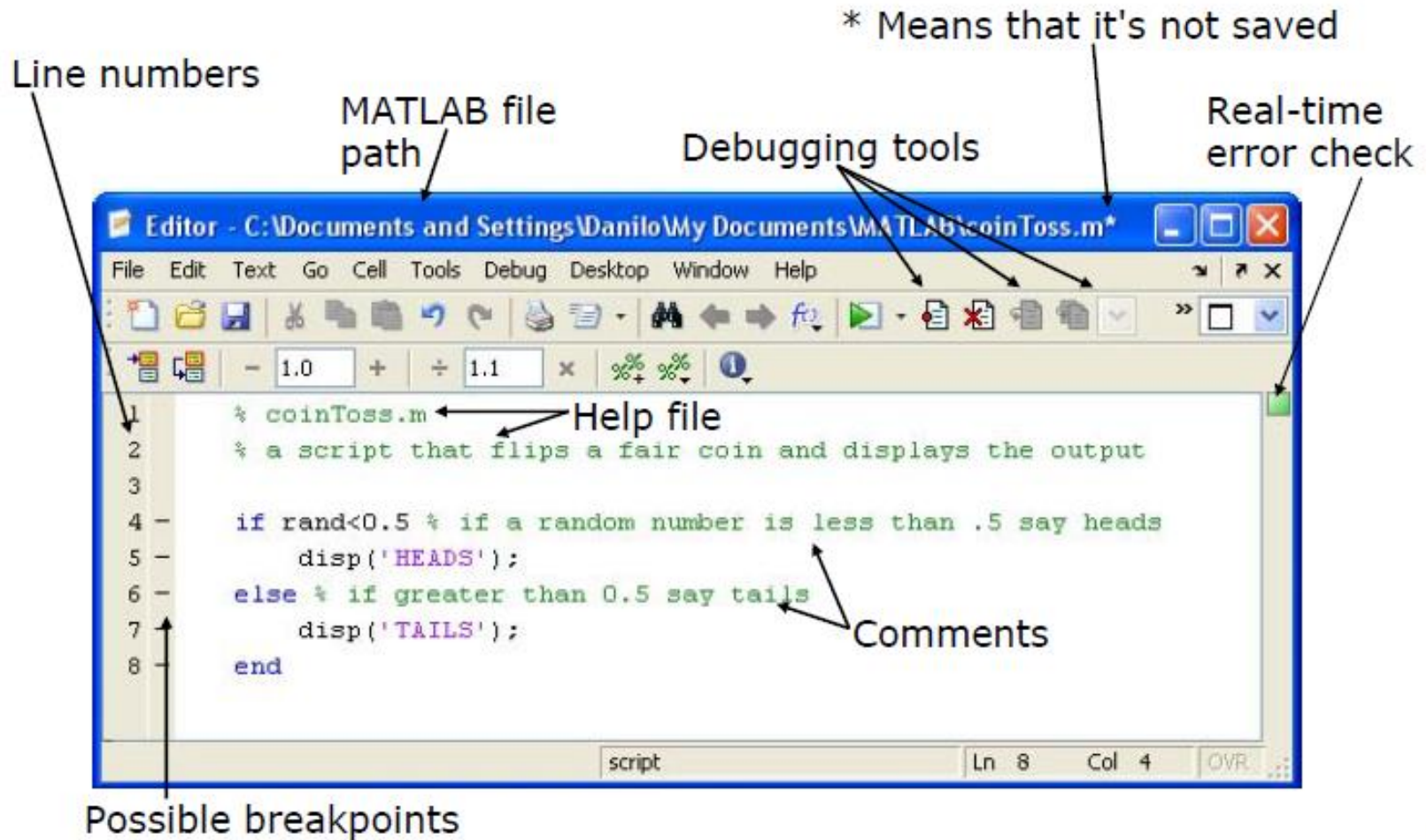
length(x)	Loop time	Find time
100	0.01	0
10,000	0.1	0
100,000	0.22	0
1,000,000	1.5	0.04

Vectorized code is more efficient for MATLAB

Use indexing and matrix operations to avoid loops

Scripting

Script files are ASCII (text) files containing MATLAB commands.



Commonly known as *m-files* because they have a .m extension

Function m-files

Example: Write a function m-file which calculates the area of a triangle having sides a , b and c using the formula: $\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}$ Where $s = (a+b+c)/2$

List of output(s) The function name. Also the name of the m-file where the function definition will be stored.

```
function [A] = area(a,b,c)
```

List of inputs

```
% Compute the area of a triangle whose  
% sides have length a, b and c.  
% Inputs:  
% a,b,c: Lengths of sides  
% Output:  
% A: area of triangle  
s = (a+b+c)/2;  
A = sqrt(s*(s-a)*(s-b)*(s-c));  
%%%%%%%% end of area %%%%%%%%%
```

Purpose of the function and how it can be used. Mainly to aid the future users

The code

File Input-Output

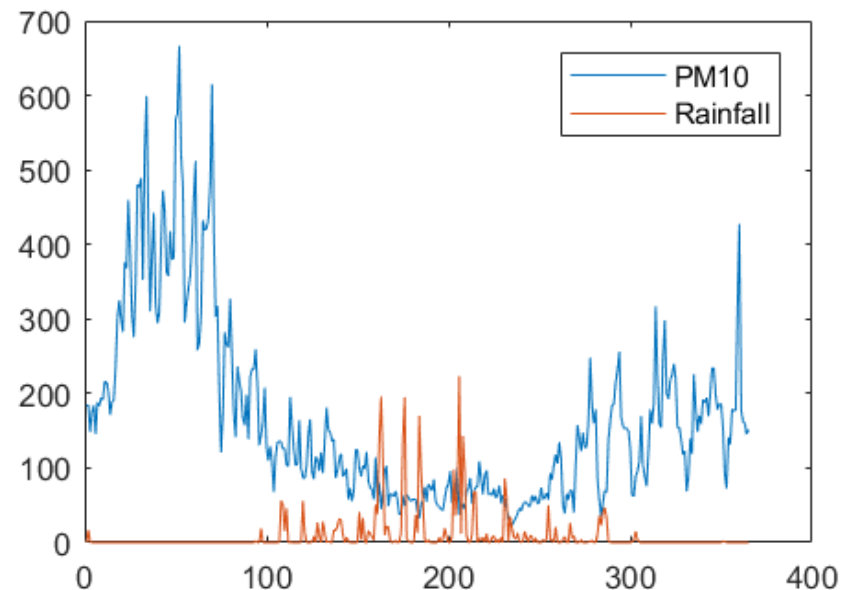
csvread

- imports numeric data with comma-separated values (csv)

Example: The file PM10_Rainfall.csv contains the particulate pollutant and rainfall data of Dhaka City in 2018. Import the data and generate two variables, pm10 and rainfall, and plot daily PM10 and rainfall in the same figure.

```
data = csvread("PM10_Rainfall.csv") ;  
pm10 = data(:,1) ;  
rainfall = data(:,2) ;  
plot(pm10) ;  
hold on ;  
plot(rainfall) ;  
hold off ;  
legend('PM10','Rainfall') ;
```

Only works with Numerical Data



File Input-Output

`xlsread`

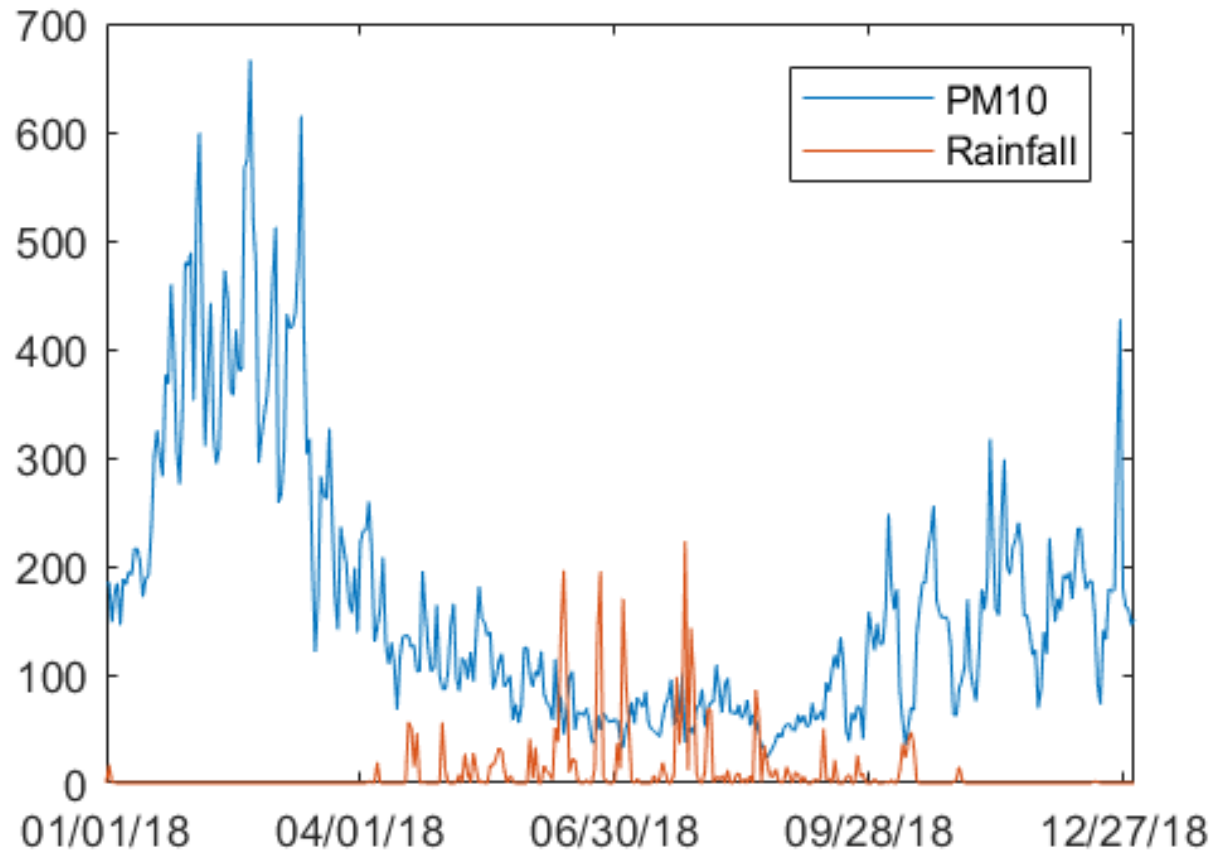
- imports excel (xls or.xlsx) format data
- both numeric and text data can be imported

Example: The file PM10_Rainfall.xlsx contains data, particulate pollutant and rainfall data of Dhaka City in 2018. Import the data and plot daily PM10 and rainfall in the same figure.

```
[num, txt] = xlsread("PM10_Rainfall.xlsx");  
pm10 = num(:,1);  
rainfall = num(:,2);  
dateVar = txt(2:end,1);  
plot(pm10); hold on;  
plot(rainfall); hold off;  
legend('PM10','Rainfall');  
set(gca,'xtick',1:90:365,...  
    'xticklabel',dateVar(1:90:365),'xlim',[1, 365]);
```

Recent versions encourage use of “readtable”, instead of “xlsread”

File Input-Output



File Input-Output

csvwrite

- export numeric variable to a comma-separated values (csv) file
- csvwrite (filename, varName)

Example:

```
A = [2 3 4 7; 5 6 8 9; 1 2 4 5];  
csvwrite('matA.csv', A);
```

xlswrite

- can export not only numeric data but also text data
- xlswrite (filename, varName)

```
A = {'Time', 'Temperature'; 9, 25; 12, 27; 3, 28; 6, 24};  
xlswrite ('time_temp.xlsx', A);
```